

PORTFOLIO R2.06

Exploitation d'une base de données

Brigade Criminelle



1. Introduction.....	2
2. Architecture technique.....	2
3. Modèle de données.....	4
4. Requêtes SQL.....	6
5. Vues SQL.....	12
6. Dashboard Grafana.....	14
7. Sauvegarde et restauration.....	16
8. Conclusion.....	18

ZYAD ABOU ELELA

BUT Informatique — 1re année

Module R2.06 — Exploitation d'une base de données

27/05/2026

1. Introduction

Ce dossier présente un projet réalisé dans le cadre du module R2.06 - Exploitation d'une base de données - de la 1re année du BUT Informatique.

Le projet s'appuie sur un scénario fictif : la gestion informatique d'une brigade criminelle qui doit suivre ses affaires, ses agents enquêteurs, les individus impliqués (suspects, témoins, victimes) et les pièces à conviction associées à chaque dossier. La base contient 5 000 affaires réparties sur plusieurs années, ce qui permet de réaliser des analyses statistiques et géographiques pertinentes.

Objectif du projet

L'objectif est de démontrer la maîtrise complète de la chaîne d'exploitation de données utilisée en entreprise :

Linux héberge les services informatiques (PostgreSQL et Grafana).

PostgreSQL stocke les données et garantit leur cohérence grâce aux contraintes relationnelles.

SQL interroge et transforme les données pour répondre à des questions métier.

Grafana transforme les résultats en tableaux de bord visuels exploitables pour prendre des décisions.

2. Architecture technique

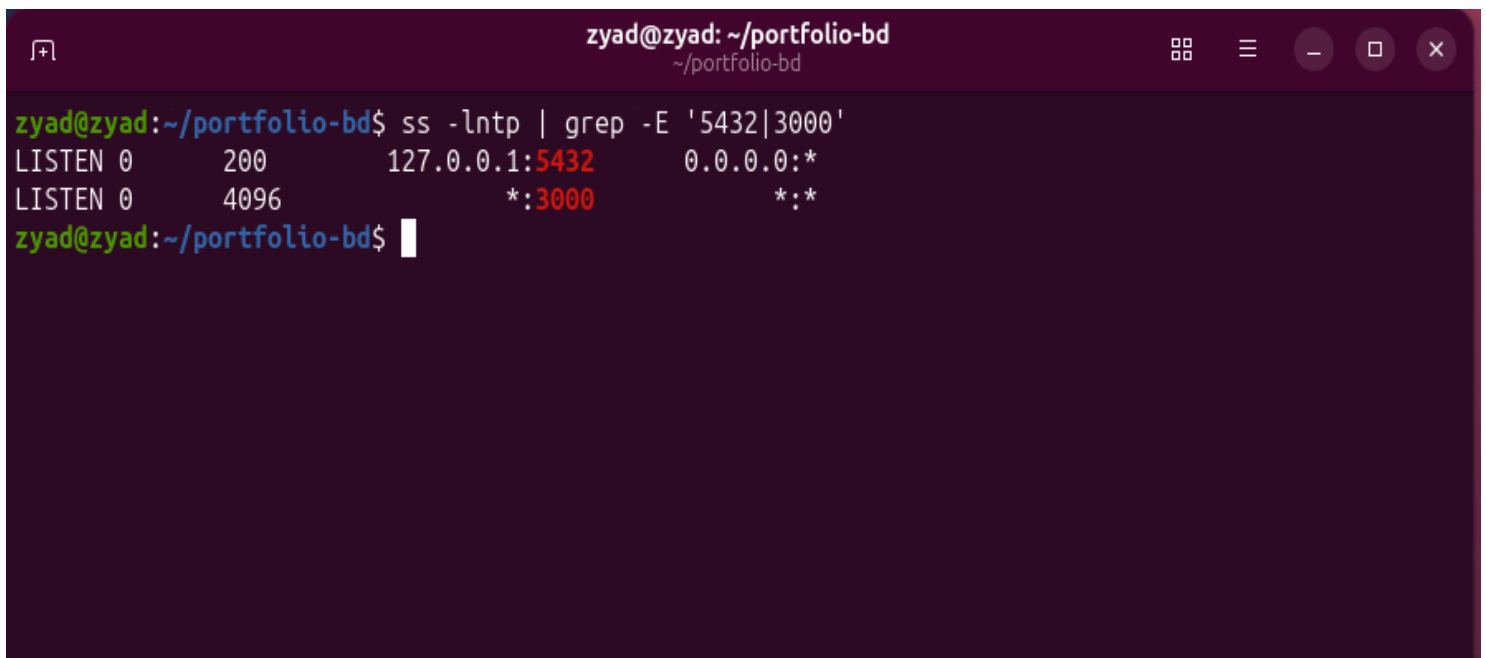
La solution s'appuie sur trois outils distincts, déployés sur une seule machine Ubuntu Linux.

PostgreSQL

Système de gestion de base de données relationnelle. Démarré comme un service Linux, il écoute par défaut sur le port 5432. C'est lui qui stocke physiquement les données et applique les contraintes d'intégrité (clés primaires, clés étrangères, types, vérifications).

Grafana

Outil de visualisation et de tableau de bord. Démarré également comme un service Linux, il écoute sur le port 3000 et expose une interface web. Grafana ne stocke aucune donnée métier : il se connecte à PostgreSQL en client SQL classique pour récupérer les résultats à afficher.



```

zyad@zyad: ~/portfolio-bd
ss -lntp | grep -E '5432|3000'
LISTEN 0      200      127.0.0.1:5432  0.0.0.0:*
LISTEN 0      4096     *:3000        *:*
```

Figure 1 — PostgreSQL (port 5432) et Grafana (port 3000) cohabitent sur le même Linux.

La capture ci-dessus prouve que les deux services tournent côte à côte : PostgreSQL sur le port 5432 (écoute restreinte à 127.0.0.1) et Grafana sur le port 3000 (écoute sur toutes les interfaces). Cette séparation des ports permet aux deux services de cohabiter sans conflit.

Distinction : utilisateur Linux et rôle PostgreSQL

Un point essentiel est qu'un utilisateur Linux et un rôle PostgreSQL sont indépendants :
Le premier appartient au système, le second à PostgreSQL.

J'ai rencontré ce point lors d'une erreur d'authentification : en tentant de me connecter avec `psql -U student`, PostgreSQL refusait avec le message « FATAL: Peer authentication failed for user student ». La cause : l'authentification par défaut en local est peer, qui vérifie que le nom de l'utilisateur Linux correspond au nom du rôle PostgreSQL. Mon utilisateur Linux étant zyad et non student, la connexion locale échouait.

La solution consiste à passer par une connexion TCP/IP avec mot de passe :

```
psql -h 127.0.0.1 -U student -d brigade_criminelle
```

Cette commande contourne le socket UNIX local et force l'utilisation de l'authentification scram-sha-256 (mot de passe).

3. Modèle de données

La base brigade_criminelle est composée de six tables relationnelles :

```

zyad@zyad: ~/portfolio-bd/vinfr206-bd-ressources — /usr/lib/postgresql/18/bin/psql -h 127...
~/portfolio-bd/vinfr206-bd-ressources
brigade_criminelle=# \dt
               List of tables
 Schema |      Name      | Type | Owner
-----+-----+-----+-----
 public | affaire        | table | student
 public | agent          | table | student
 public | departement_geo | table | student
 public | implique       | table | student
 public | individu       | table | student
 public | piece          | table | student
(6 rows)

brigade_criminelle=#

```

Figure 2 — Liste des 6 tables de la base brigade_criminelle.

Description des tables

affaire : Le coeur de la base. Chaque ligne représente un dossier criminel avec son statut (ouverte, en_cours, close), son type (vol, agression, homicide...), ses dates de début et de fin, et le département où elle s'est produite.

agent : Les enquêteurs de la brigade. Lié à individu par num_individu (un agent est un individu doté d'un matricule et éventuellement d'un supérieur hiérarchique).

individu : Toutes les personnes connues de la brigade (agents, suspects, témoins, victimes). Stocke nom, prénom et la présence d'un casier judiciaire.

implique : Table de liaison entre individus et affaires, avec un rôle (témoin, suspect, victime, enquêteur). C'est par cette table qu'on relie un agent à ses dossiers.

piece : Les pièces à conviction associées à chaque dossier.

departement_geo : Table géographique contenant les coordonnées (latitude, longitude) de

chaque département français. Utilisée pour la cartographie Grafana.

```

brigade_criminelle=# \d affaire
Table "public.affaire"
  Column      |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
dossier       | character varying(20) |           | not null |
statut        | character varying(20) |           | not null |
type          | character varying(50) |           | not null |
date_debut    | date              |           | not null |
date_fin      | date              |           |          |
departement_code | character varying(3) |           | not null |
Indexes:
    "affaire_pkey" PRIMARY KEY, btree (dossier)
    "idx_affaire_date_debut" btree (date_debut)
    "idx_affaire_departement_code" btree (departement_code)
    "idx_affaire_statut" btree (statut)
    "idx_affaire_type" btree (type)
Check constraints:
    "chk_affaire_dates" CHECK (date_fin IS NULL OR date_fin >= date_debut)
    "chk_affaire_statut" CHECK (statut::text = ANY (ARRAY['ouverte'::character varying, 'en_cours'::character varying, 'close'::character varying]::text[]))
    "chk_departement_code_non_vide" CHECK (btrim(departement_code::text) <> ''::text)
Referenced by:
    TABLE "implique" CONSTRAINT "fk_implique_affaire" FOREIGN KEY (dossier) REFERENCES affaire(dossier) ON DELETE CASCADE
    TABLE "piece" CONSTRAINT "fk_piece_affaire" FOREIGN KEY (dossier) REFERENCES affaire(dossier) ON DELETE CASCADE
brigade_criminelle=#

```

Figure 3 — Structure détaillée de la table affaire.

La capture de la table affaire met en évidence trois éléments clés :

La clé primaire : dossier, qui identifie chaque affaire de manière unique.

Les contraintes CHECK : le statut doit appartenir à la liste {ouverte, en_cours, close}, et date_fin doit être supérieure ou égale à date_debut.

Les clés étrangères entrantes (Referenced by) : les tables implique et piece référencent affaire(dossier). La clause ON DELETE CASCADE garantit qu'en supprimant une affaire, ses pièces et implications associées disparaissent automatiquement.

Cette structure normalisée évite la redondance (chaque information n'est stockée qu'une fois) et garantit la cohérence (impossible de créer une pièce ou une implication sans affaire existante).

Schéma relationnel :

affaire (dossier, statut, type, date_debut, date_fin, *departement_code*)

- *departement_code* est une clé étrangère qui fait référence à `departement_geo(code_departement)`

individu (num_individu, nom, prenom, casier)

agent (num_individu, matricule, *matricule_superieur*)

- *num_individu* est une clé étrangère qui fait référence à `individu(num_individu)`
- *matricule_superieur* est une clé étrangère qui fait référence à `agent(matricule)` (*auto-référence*)
- *matricule* porte une contrainte d'unicité (UNIQUE)

implique (num_individu, dossier, *role*)

- *num_individu* est une clé étrangère qui fait référence à `individu(num_individu)`
- *dossier* est une clé étrangère qui fait référence à `affaire(dossier)`
- Table de liaison réalisant une relation N:N entre `individu` et `affaire`, qualifiée par *role* (suspect, témoin, victime, enquêteur)

piece (num_piece, description, *dossier*)

- *dossier* est une clé étrangère qui fait référence à `affaire(dossier)`
- *description* décrit la nature de la pièce à conviction

departement_geo (code_departement, nom_departement, latitude, longitude)

4. Requêtes SQL

Six requêtes SQL ont été conçues pour explorer la base sous différents angles. Chacune mobilise des notions spécifiques (jointure, agrégation, sous-requête, vue) et apporte une information métier différente.

Requête 1 — Répartition des affaires par type et statut

Question : *Quel est le mix d'activité de la brigade ? Quels types d'enquêtes dominent et dans quel état d'avancement sont-elles ?*

```
SELECT type, statut, COUNT(*) AS nombre
FROM affaire
GROUP BY type, statut
ORDER BY type, statut;
```

```
brigade_criminelle=# SELECT type, statut, COUNT(*) AS nombre
FROM affaire
GROUP BY type, statut
ORDER BY type, statut;
```

type	statut	nombre
agression	close	143
agression	en_cours	429
agression	ouverte	428
association_de_malfaiteurs	close	29
association_de_malfaiteurs	en_cours	85
association_de_malfaiteurs	ouverte	86
cybercriminalite	close	86
cybercriminalite	en_cours	257
cybercriminalite	ouverte	257
disparition	close	57
disparition	en_cours	172
disparition	ouverte	171
escroquerie	close	107
escroquerie	en_cours	343
escroquerie	ouverte	300
homicide	close	7
homicide	ouverte	43
trafic	close	71
trafic	en_cours	214
trafic	ouverte	215
vol	close	214
vol	en_cours	643
vol	ouverte	643

(23 rows)

Figure 4 — Répartition des 5 000 affaires par type et statut.

Interprétation

Le résultat (23 lignes) révèle que les vols dominent largement avec 1 500 affaires au total, suivis par les agressions (1 000) et les escroqueries (750). Les homicides sont les moins fréquents (50 affaires).

Pour les vols, on observe un équilibre entre affaires ouvertes (643) et en cours (643), pour seulement 214 affaires closes.

Requête 2 — Top 5 des enquêteurs les plus actifs

Question : *Quels enquêteurs gèrent le plus de dossiers ? Identifier une éventuelle surcharge.*

```
SELECT i.nom,
       i.prenom,
       ag.matricule,
       COUNT(DISTINCT imp.dossier) AS nb_affaires
FROM agent ag
JOIN individu i ON i.num_individu = ag.num_individu
JOIN implique imp ON imp.num_individu = ag.num_individu
                  AND imp.role = 'enqueteur'
GROUP BY ag.matricule, i.nom, i.prenom
ORDER BY nb_affaires DESC
LIMIT 5;
```

Figure 5 — Top 5 des enquêteurs les plus actifs.

Interprétation

La requête combine trois tables (agent, individu, implique) pour récupérer les noms des enquêteurs et compter les affaires sur lesquelles ils sont intervenus, filtrées par le rôle 'enqueteur'. Le résultat montre que les cinq premiers gèrent chacun 20 affaires, ce qui suggère une répartition de la charge équilibrée au moins parmi le top du classement.

Requête 3 — Individus impliqués dans plusieurs affaires

Question : *Quelles personnes apparaissent dans plusieurs dossiers ? Profils à surveiller (récidive, témoins multiples, victimes répétées).*

```
SELECT i.nom,
       i.prenom,
       i.casier,
       COUNT(DISTINCT imp.dossier) AS nb_affaires
FROM individu i
JOIN implique imp ON imp.num_individu = i.num_individu
WHERE imp.role IN ('suspect', 'temoin', 'victime')
GROUP BY i.num_individu, i.nom, i.prenom, i.casier
HAVING COUNT(DISTINCT imp.dossier) > 1
ORDER BY nb_affaires DESC
LIMIT 20;
```



```

nom | prenom | casier | nb_affaires
-----
zyad@zyad: ~/portfolio-bd/vinfr206-bd-ressources — /usr/lib/postgresql/18/bin/psql -h 127....
~/portfolio-bd/vinfr206-bd-ressources

brigade_criminelle=# SELECT i.nom,
                        i.prenom,
                        ag.matricule,
                        COUNT(DISTINCT imp.dossier) AS nb_affaires
FROM agent ag
JOIN individu i ON i.num_individu = ag.num_individu
JOIN implique imp ON imp.num_individu = ag.num_individu
                  AND imp.role = 'enqueteur'
GROUP BY ag.matricule, i.nom, i.prenom
ORDER BY nb_affaires DESC
LIMIT 5;
 nom | prenom | matricule | nb_affaires
-----+-----+-----+-----
Nom2 | Prenom2 | AG0002    |          20
Nom3 | Prenom3 | AG0003    |          20
Nom4 | Prenom4 | AG0004    |          20
Nom5 | Prenom5 | AG0005    |          20
Nom1 | Prenom1 | AG0001    |          20
(5 rows)

brigade_criminelle=#

```

Figure 6 — Individus impliqués dans plusieurs affaires.

Interprétation

La clause `HAVING` permet de filtrer après agrégation : on garde uniquement les individus apparaissant dans plus d'une affaire. Le résultat liste des individus impliqués dans 5 affaires en tant que suspect, témoin ou victime. La colonne `casier` (t/f) indique la présence d'un casier judiciaire, donnée utile pour orienter une éventuelle enquête.

Le résultat révèle une distribution intéressante : tous les individus en tête du classement apparaissent exactement dans 5 dossiers, sans intermédiaire. En diagnostiquant la donnée plus en profondeur, j'ai constaté que le générateur de la base a créé deux populations : une majorité d'individus impliqués dans 1 seule affaire, et un petit groupe d'individus systématiquement impliqués dans 5 affaires. Dans un contexte réel, ces profils correspondraient à des "récidivistes" ou à des témoins/victimes récurrents qu'il faudrait suivre de près.

Requête 4 — Mois d'activité supérieurs à la moyenne

Question : *Quels mois ont vu une activité criminelle anormalement élevée ? Détection de pics qui méritent investigation.*

```
SELECT TO_CHAR(date_debut, 'YYYY-MM') AS mois,
       COUNT(*) AS nb_affaires
FROM affaire
GROUP BY mois
HAVING COUNT(*) > (
    SELECT AVG(nb_mois) FROM (
        SELECT COUNT(*) AS nb_mois
        FROM affaire
        GROUP BY TO_CHAR(date_debut, 'YYYY-MM')
    ) AS moyenne_mensuelle
)
ORDER BY mois;
```

mois	nb_affaires
2022-07	112
2022-08	115
2023-01	113
2023-02	106
2023-03	119
2023-04	125
2023-05	127
2023-06	121
2023-07	117
2023-08	115
2023-09	105
2023-10	108
2023-12	118
2024-01	124
2024-02	114
2024-03	133
2024-04	128
2024-05	123
2024-06	115
2024-07	108
2024-11	116
2024-12	128

(22 rows)

Figure 7 — Mois d'activité supérieurs à la moyenne.

Interprétation

La sous-requête imbriquée calcule d'abord la moyenne mensuelle d'affaires sur toute la période. La requête principale ne conserve ensuite que les mois dépassant cette moyenne. Les résultats montrent des pics d'activité en 2023-2024 (jusqu'à 133 affaires en mars 2024),

traduisant des périodes de forte tension qu'il serait intéressant de corrélérer avec des événements externes (vacances scolaires, manifestations, etc.).

Requête 5 — Synthèse par type via une vue SQL

Question métier : Vue d'ensemble synthétique de l'activité par type, avec taux de résolution.

```
SELECT type,
       total,
       ouvertes,
       en_cours,
       closes,
       ROUND(100.0 * closes / total, 1) AS taux_resolution_pct
FROM synthese_affaire
ORDER BY total DESC;
```

type	total	ouvertes	en_cours	closes	taux_resolution_pct
vol	1500	643	643	214	14.3
agression	1000	428	429	143	14.3
escroquerie	750	300	343	107	14.3
cybercriminalite	600	257	257	86	14.3
trafic	500	215	214	71	14.2
disparition	400	171	172	57	14.3
association_de_malfaiteurs	200	86	85	29	14.5
homicide	50	43	0	7	14.0

(8 rows)

Figure 8 — Synthèse via la vue synthese_affaire.

Interprétation

Cette requête utilise la vue synthese_affaire créée préalablement (voir section 5). Le code SQL n'occupe que sept lignes alors que la requête équivalente sans vue en ferait une dizaine. C'est l'intérêt majeur des vues SQL : factoriser la complexité. Le calcul du taux de résolution (pourcentage de close / total) met en évidence les types les plus difficiles à clôturer.

Notions mobilisées : utilisation d'une vue SQL, calcul de pourcentage avec ROUND.

Requête 6 — Répartition géographique

Question métier : Quels départements concentrent le plus d'activité criminelle ?

```
SELECT d.code_departement,
       d.nom_departement,
       COUNT(a.dossier) AS total_affaires
FROM departement_geo d
```

```

LEFT JOIN affaire a ON a.departement_code = d.code_departement
GROUP BY d.code_departement, d.nom_departement
ORDER BY total_affaires DESC
LIMIT 15;

```

code_departement	nom_departement	total_affaires
31	Haute-Garonne	354
67	Bas-Rhin	289
69	Rhone	254
06	Alpes-Maritimes	253
75	Paris	250
13	Bouches-du-Rhone	250
93	Seine-Saint-Denis	222
33	Gironde	173
59	Nord	142
44	Loire-Atlantique	130
02	Aisne	54
76	Seine-Maritime	53
61	Orne	53
48	Lozere	53
63	Puy-de-Dome	53

(15 rows)

brigade_criminelle=#

Figure 9 — Répartition par département (top 15).

Interprétation

Le LEFT JOIN permet d'inclure tous les départements même ceux n'ayant aucune affaire — utile pour ne pas masquer les zones « blanches ». Les départements en tête du classement correspondent généralement aux zones les plus densément peuplées (Paris, Bouches-du-Rhône, Nord, Rhône), confirmant la corrélation classique entre densité démographique et volume d'affaires criminelles.

Synthèse des notions couvertes

Les six requêtes couvrent collectivement l'ensemble des notions exigées par le sujet :

Jointures : requêtes 2, 3, 5, 6

Agrégations avec GROUP BY : toutes les requêtes.

HAVING : requêtes 3 et 4.

Sous-requête : requête 4 (imbriquée à deux niveaux).

Vue SQL : requête 5.

5. Vues SQL

Pour simplifier les requêtes répétitives et préparer les données destinées à Grafana, j'ai créé deux vues SQL distinctes :

Vue 1 — synthese_affaire

Cette vue agrège le nombre d'affaires par type, avec ventilation par statut. Elle est utilisée pour le panel Bar chart du dashboard et pour la requête 5 (section 4).

```
CREATE OR REPLACE VIEW synthese_affaire AS
SELECT
    type,
    COUNT(*) AS total,
    COUNT(*) FILTER (WHERE statut = 'ouverte') AS ouvertes,
    COUNT(*) FILTER (WHERE statut = 'en_cours') AS en_cours,
    COUNT(*) FILTER (WHERE statut = 'close') AS closes
FROM affaire
GROUP BY type;
```

Vue 2 — affaires_par_departement

Cette deuxième vue pré-calcule le nombre d'affaires par département en joignant affaire avec departement_geo. Elle est utilisée directement par le panel Geomap (carte de France).

```
CREATE OR REPLACE VIEW affaires_par_departement AS
SELECT
    d.code_departement,
    d.nom_departement,
    d.latitude,
    d.longitude,
    COUNT(a.dossier) AS total_affaires
FROM departement_geo d
LEFT JOIN affaire a ON a.departement_code = d.code_departement
GROUP BY d.code_departement, d.nom_departement, d.latitude,
d.longitude;
```

```

zyad@zyad:~/portfolio-bd/vinfr206-bd-ressources$ cd ~/portfolio-bd/vinfr206-bd-ressources
psql -h 127.0.0.1 -U student -d brigade_criminelle
Password for user student:
psql (18.4 (Ubuntu 18.4-0ubuntu0.26.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql
)
Type "help" for help.

brigade_criminelle=# -- Vue 1 : synthèse des affaires par type, avec ventilation par statut
CREATE OR REPLACE VIEW synthese_affaire AS
SELECT
    type,
    COUNT(*) AS total,
    COUNT(*) FILTER (WHERE statut = 'ouverte') AS ouvertes,
    COUNT(*) FILTER (WHERE statut = 'en_cours') AS en_cours,
    COUNT(*) FILTER (WHERE statut = 'close') AS closes
FROM affaire
GROUP BY type;

-- Vue 2 : pré-agrégat géographique (utilisée par la Geomap Grafana)
CREATE OR REPLACE VIEW affaires_par_departement AS
SELECT
    d.code_departement,
    d.nom_departement,
    d.latitude,
    d.longitude,
    COUNT(a.dossier) AS total_affaires
FROM departement_geo d
LEFT JOIN affaire a ON a.departement_code = d.code_departement
GROUP BY d.code_departement, d.nom_departement, d.latitude, d.longitude;
CREATE VIEW
CREATE VIEW
brigade_criminelle=# █

```

Figure 10 — Création des deux vues SQL.

Intérêt des vues SQL

Les vues offrent trois avantages majeurs par rapport à des requêtes brutes répétées :

Factorisation : la définition de la requête est centralisée. Si la logique métier change, je modifie la vue, et tous les dashboards qui l'utilisent sont mis à jour automatiquement.

Lisibilité : `SELECT * FROM synthese_affaire` est beaucoup plus parlant qu'un `GROUP BY` de cinq lignes recopié partout.

Cohérence : tous les panels qui utilisent la même vue affichent une donnée identique, par construction.

Point important que j'ai compris : une vue ne stocke pas physiquement les données. Elle stocke uniquement la définition de la requête, qui est ré-exécutée à chaque appel. Cela

garantit des résultats toujours à jour, mais peut être plus coûteux que des données pré-calculées (vues matérialisées) sur de très grosses bases.

6. Dashboard Grafana

Le dashboard Grafana combine cinq visualisations complémentaires pour offrir une lecture rapide et complète de l'activité criminelle.

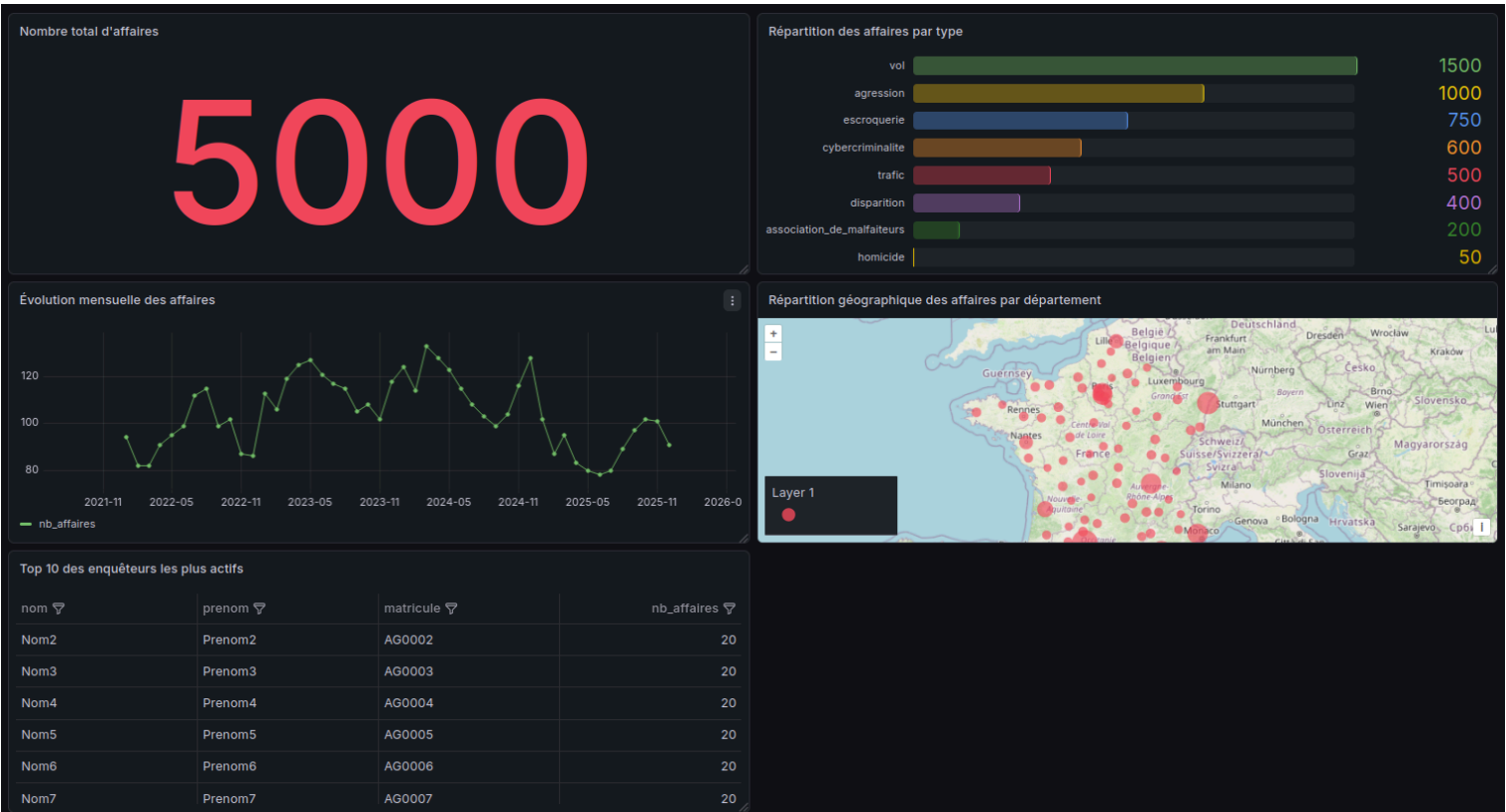


Figure 11 — Dashboard complet avec les cinq panels.

Détail des panels

Panel 1 — Indicateur clé (Stat)

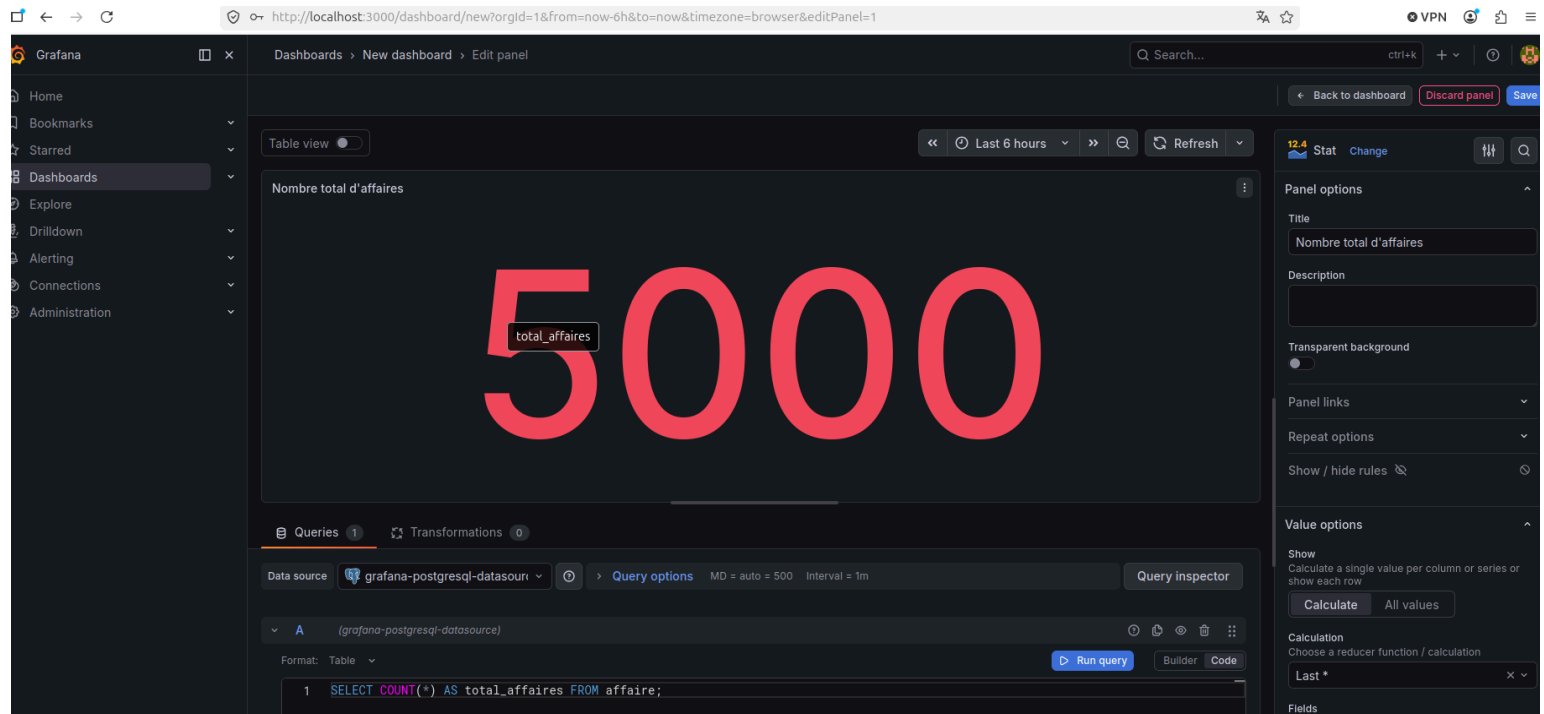


Figure 12 — Panel Stat : nombre total d'affaires.

Affiche le nombre total d'affaires en très grands caractères. Le type de visualisation « Stat » est idéal pour un indicateur clé qu'on veut mettre en avant immédiatement, sans concurrent visuel. Une simple cellule dans un tableau aurait été bien moins impactante.

Panel 2 — Répartition par type (Bar chart horizontal)

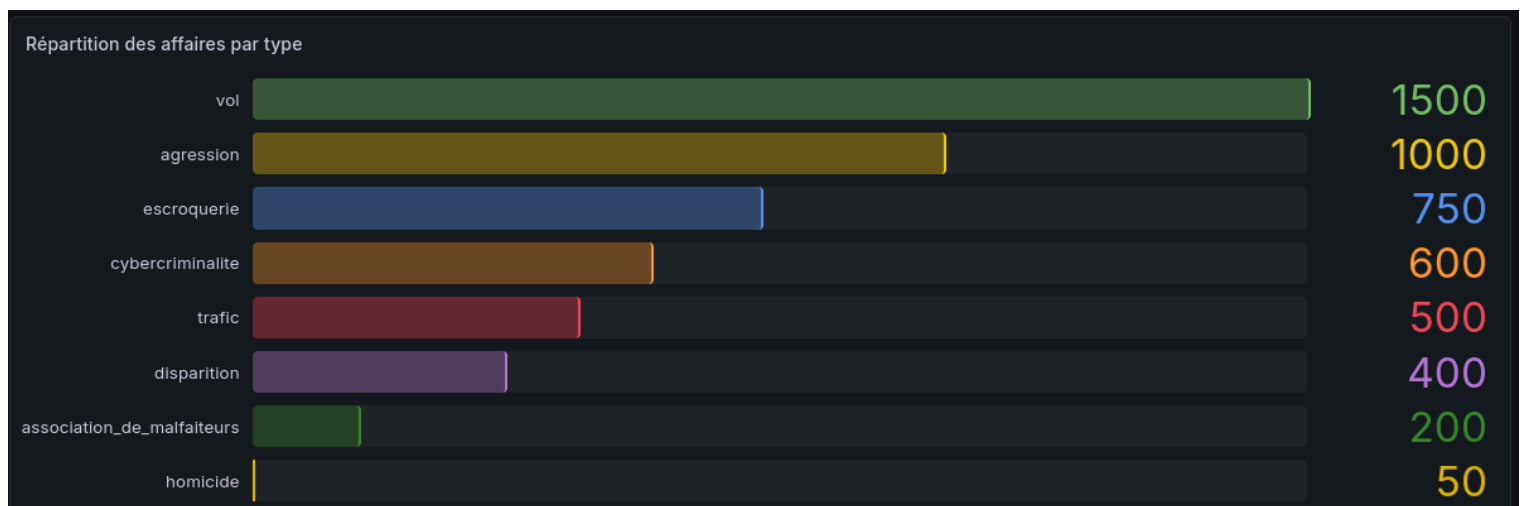


Figure 13 — Bar chart horizontal des types d'affaires.

J'ai choisi un Bar chart horizontal plutôt qu'un Pie chart. Avec huit catégories d'affaires, un camembert aurait produit des parts trop fines pour la lecture des plus petites valeurs (homicide à 50, association de malfaiteurs à 200). Le Bar chart permet une comparaison directe et un classement immédiat des types par fréquence.

Panel 3 — Évolution mensuelle (Time series)

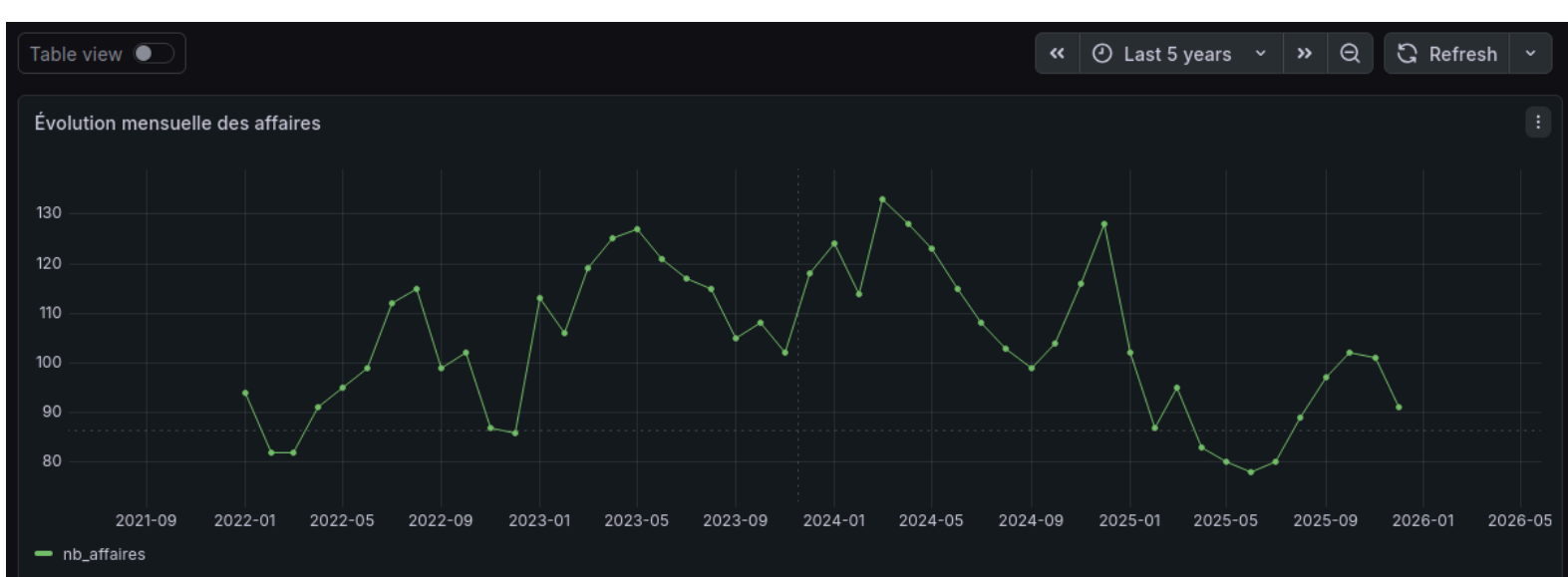


Figure 14 — Évolution mensuelle des affaires.

La série temporelle révèle des oscillations entre 80 et 130 affaires par mois sur quatre années. C'est cette visualisation qui permet de détecter des tendances et des pics que les données brutes ne feraient jamais ressortir. Un tableau aurait simplement aligné des nombres sans révéler de motif.

Panel 4 — Top 10 enquêteurs (Table)

Table view ☐

<< ⌚ Last 5 years >> 🔍 ↻ Refresh ▾

Top 10 des enquêteurs les plus actifs ⋮

nom 📄	prenom 📄	matricule 📄	nb_affaires 📄
Nom2	Prenom2	AG0002	20
Nom3	Prenom3	AG0003	20
Nom4	Prenom4	AG0004	20
Nom5	Prenom5	AG0005	20
Nom6	Prenom6	AG0006	20
Nom7	Prenom7	AG0007	20
Nom8	Prenom8	AG0008	20
Nom9	Prenom9	AG0009	20
Nom10	Prenom10	AG0010	20
Nom1	Prenom1	AG0001	20

Figure 15 — Top 10 des enquêteurs les plus actifs.

Quand l'information est essentiellement textuelle (noms, prénoms, matricules), la table reste le format le plus adapté. Un Bar chart aurait été possible pour le nombre d'affaires, mais le tableau permet de conserver toutes les informations d'identité lisibles d'un coup d'œil.

Panel 5 — Répartition géographique (Geomap)

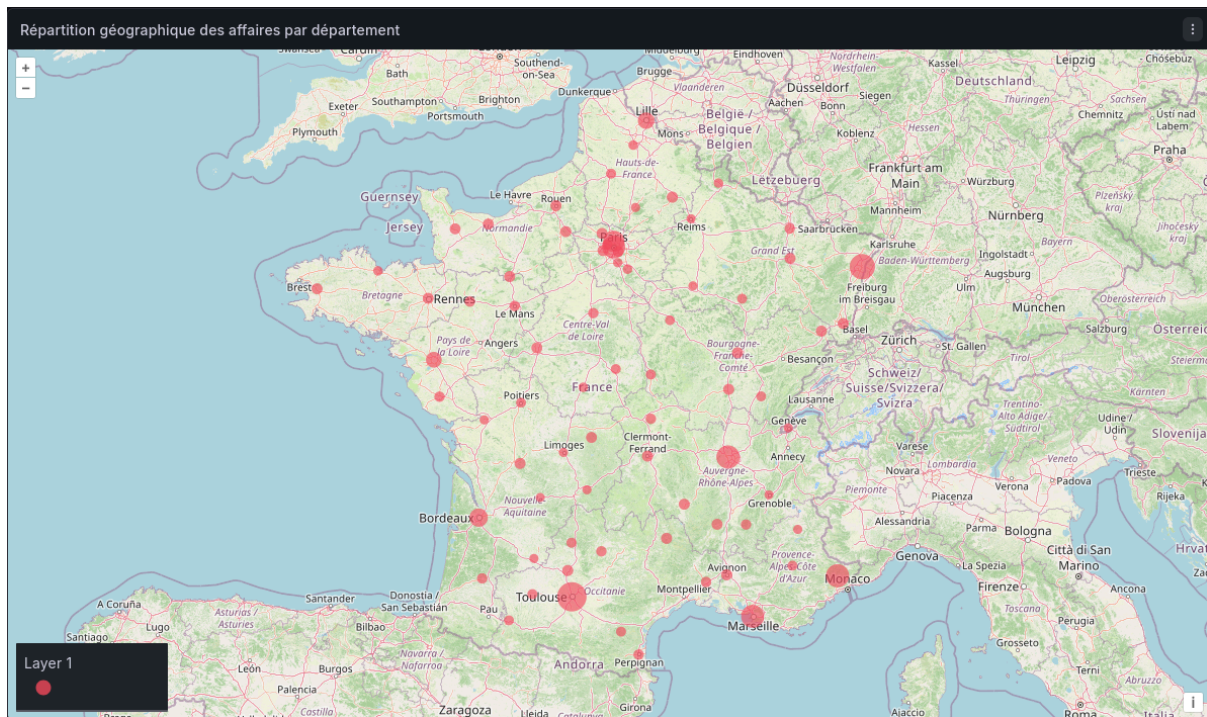


Figure 16 — Geomap des affaires par département.

Le panel le plus visuel du dashboard. La carte de France permet de saisir d'un coup d'œil les zones d'activité: quelque chose qu'une liste triée de départements ne pourrait jamais transmettre. La taille des cercles est proportionnelle au nombre d'affaires : Paris, le Rhône et les Bouches-du-Rhône concentrent visiblement l'activité.

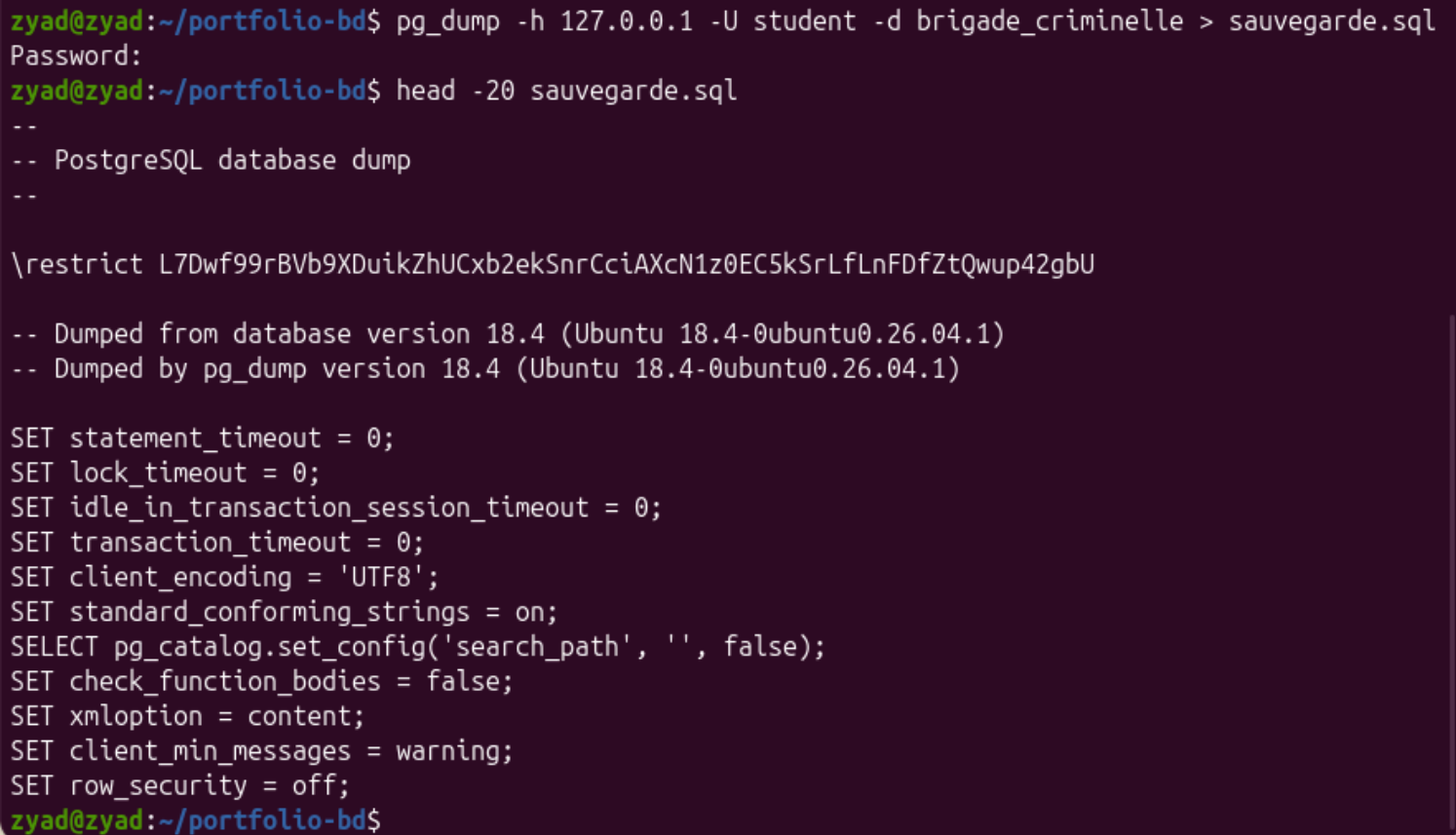
Grafana et PostgreSQL : qui fait quoi

Important pour comprendre l'architecture : Grafana ne stocke aucune donnée métier. À chaque rafraîchissement du dashboard, il envoie les requêtes SQL au serveur PostgreSQL et récupère les résultats. Cela garantit que les graphiques sont toujours à jour par rapport à la base, et que la séparation entre stockage (PostgreSQL) et visualisation (Grafana) est nette.

7. Sauvegarde et restauration

La sauvegarde de la base est réalisée avec l'outil `pg_dump` fourni par PostgreSQL :

```
pg_dump -h 127.0.0.1 -U student -d brigade_criminelle > sauvegarde.sql
```



```
zyad@zyad:~/portfolio-bd$ pg_dump -h 127.0.0.1 -U student -d brigade_criminelle > sauvegarde.sql
Password:
zyad@zyad:~/portfolio-bd$ head -20 sauvegarde.sql
--
-- PostgreSQL database dump
--

\restrict L7Dwf99rBVb9XDuikZhUCxb2ekSnrCciAXcN1z0EC5kSrLfLnFDfZtQwup42gbU

-- Dumped from database version 18.4 (Ubuntu 18.4-0ubuntu0.26.04.1)
-- Dumped by pg_dump version 18.4 (Ubuntu 18.4-0ubuntu0.26.04.1)

SET statement_timeout = 0;
SET lock_timeout = 0;
SET idle_in_transaction_session_timeout = 0;
SET transaction_timeout = 0;
SET client_encoding = 'UTF8';
SET standard_conforming_strings = on;
SELECT pg_catalog.set_config('search_path', '', false);
SET check_function_bodies = false;
SET xmloption = content;
SET client_min_messages = warning;
SET row_security = off;
zyad@zyad:~/portfolio-bd$
```

Figure 17 — Génération de la sauvegarde logique.

Contenu du fichier généré

Le fichier `.sql` contient tout le nécessaire pour reconstruire la base à l'identique :

Les CREATE TABLE (structure des tables)

Les CREATE INDEX (index pour la performance)

Les INSERT (toutes les données)

Les CREATE VIEW (vues SQL)

Les contraintes (clés primaires, clés étrangères, CHECK)

C'est une sauvegarde logique (au format SQL texte), à opposer à une sauvegarde physique qui copierait directement les fichiers binaires de `/var/lib/postgresql/`.

Pourquoi une telle portabilité ?

Le fichier étant du SQL texte standard, il peut être restauré sur n'importe quelle version de PostgreSQL postérieure, sur n'importe quel système d'exploitation (Linux, Windows, macOS), et sur une machine différente. C'est ce qui rend ce type de sauvegarde universel et indépendant de l'environnement de production.

Procédure de restauration

```
dropdb -h 127.0.0.1 -U student brigade_criminelle
createdb -h 127.0.0.1 -U student brigade_criminelle
psql -h 127.0.0.1 -U student -d brigade_criminelle < sauvegarde.sql
```

Enjeux de confidentialité

Pour une vraie brigade criminelle, ce fichier contiendrait les noms réels de suspects, témoins et victimes, leurs casiers judiciaires, les dates et lieux des affaires. C'est un document hautement sensible qui doit être :

Chiffré au repos (par exemple avec gpg ou un disque LUKS)

Transféré par canal sécurisé (scp et jamais par mail)

Stocké avec un contrôle d'accès strict (seuls les DBA habilités peuvent y accéder)

Détruit de façon sécurisée quand il n'est plus utile (effacement cryptographique)

Une sauvegarde mal protégée représente un risque plus important que la base de production elle-même, car elle est souvent moins surveillée.

8. Conclusion

Compétences acquises

Ce projet m'a permis de mobiliser concrètement les compétences enseignées en R2.06 :

PostgreSQL : installation et démarrage du service, création de rôles, gestion des bases.

SQL avancé : jointures multi-tables, agrégations avec FILTER, sous-requêtes imbriquées, vues SQL, fonctions de date.

Grafana : configuration de datasource PostgreSQL, création de panels variés (Stat, Bar chart, Time series, Table, Geomap), export JSON.

Administration Linux : authentification (peer / scram-sha-256), gestion des ports, services systemd, sauvegardes pg_dump.

Difficultés rencontrées:

L'erreur d'authentification PostgreSQL au début du TP a été particulièrement instructive : sans comprendre la distinction utilisateur Linux / rôle PostgreSQL, je n'aurais pas pensé à basculer en TCP/IP avec mot de passe (-h 127.0.0.1) pour résoudre le problème.

La configuration de la Geomap Grafana a été particulièrement délicate : par défaut la carte s'affichait centrée sur le monde entier avec des points minuscules, et il a fallu paramétrer manuellement les coordonnées du centre, le zoom initial, le champ de taille des marqueurs et le fond de carte clair pour obtenir un rendu lisible et exploitable.

Pour conclure,

Ce projet m'a fait travailler ensemble Linux, PostgreSQL, SQL et Grafana sur une base de 5 000 affaires : installer, interroger, visualiser et sauvegarder. J'ai surtout appris à toujours regarder les données avant d'écrire une requête, et à bien distinguer le compte Linux du rôle PostgreSQL — deux choses qui m'ont coûté du temps en début de TP. Au-delà de la note, ce dossier me sert de preuve concrète à montrer à un recruteur quand je chercherai mon stage.